

Task 5 Code Optimization

Prompt :

Optimize the following Python code by reducing its time complexity. Explain the original complexity, the optimized approach, and provide the improved code with comments.

Code :

```
arr = [1, 2, 3, 2, 4, 1]
```

```
duplicates = []
```

```
for i in range(len(arr)):
    for j in range(i + 1, len(arr)):
        if arr[i] == arr[j] and arr[i] not in duplicates:
            duplicates.append(arr[i])
```

```
print(duplicates)
```

Output :

Original Time Complexity

The original code uses **two nested loops**:

```
Python
for i in range(len(arr)):
    for j in range(i + 1, len(arr)):
```

- The outer loop runs n times.
- The inner loop runs up to $n - i - 1$ times.
- This results in approximately $n \times n$ comparisons in the worst case.

Additionally:

```
Python
arr[i] not in duplicates
```

checks membership in a list, which takes $O(k)$ time (where k is the number of duplicates found). Although this is usually smaller than n , the dominant cost remains the nested loops.

- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(d)$, where d is the number of duplicate elements.

Optimized Approach

Instead of comparing every element with every other element, use a **set** to keep track of elements that have already been seen.

Algorithm

1. Create an empty **seen** set.
2. Create another **duplicates** set.
3. Traverse the array once:
 - If the element is already in **seen**, add it to **duplicates**.
 - Otherwise, add it to **seen**.
4. Convert the duplicates set to a list (if needed).

Since set lookup and insertion are $O(1)$ on average, the entire algorithm requires only one pass through the array.

- Time Complexity: $O(n)$
- Space Complexity: $O(n)$

Optimized Code

```
</> Python  

arr = [1, 2, 3, 2, 4, 1]

seen = set()      # Stores elements we've already encountered
duplicates = set() # Stores duplicate elements

for num in arr:
    if num in seen:
        duplicates.add(num) # Add duplicate to the set
    else:
        seen.add(num)       # First occurrence of the element

# Convert to list if a list output is required
print(list(duplicates))
```

Output

```
</> Python  

[1, 2]
```

Note: Since `duplicates` is a set, the order of elements is not guaranteed. If preserving the order of first duplicate occurrence is important, use the following version:

```
</> Python  

arr = [1, 2, 3, 2, 4, 1]

seen = set()
```

```
</> Python  

duplicates = []

for num in arr:
    if num in seen:
        if num not in duplicates:
            duplicates.append(num)
    else:
        seen.add(num)

print(duplicates)
```

Output:

```
</> Python  

[2, 1]
```

This version preserves the order in which duplicates are first detected while still avoiding the expensive nested loops. Although checking `num not in duplicates` is a list lookup, it is performed only when a duplicate is encountered, making the overall approach much more efficient than the original nested-loop solution in typical cases.